

Gerenciamento de Configuração

Prof. Breno de França

Prof. Bruno Cafeo

`cafeo@unicamp.br`

Introdução

Quando um software é lançado, é comum identificarmos itens como:

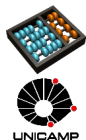
- Binários ou executável
- Código-fonte (software livre)
- Número da versão
 - Ex: 1.3, 18.04, 10.3.56
 - Versionamento Semântico (ver semver.org)
- Informações ou notas da liberação (release)
 - Features adicionadas, bugs conhecidos, entre outras
- ...

Introdução

Mas como se chega a isso?

Bom, primeiro, é trivial dizer: “pelo processo de desenvolvimento!”

Certo, mas quais atividades estão envolvidas?

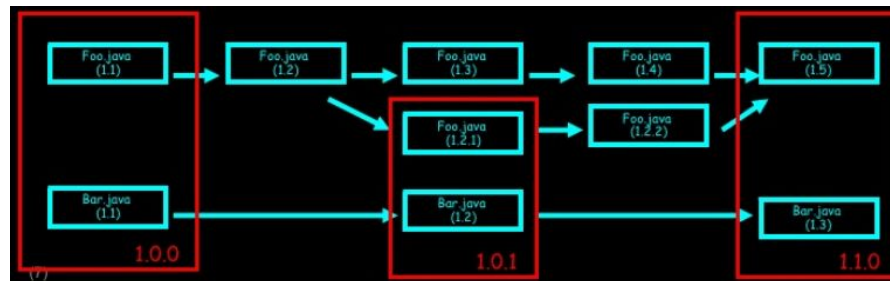


Introdução

No processo de desenvolvimento, temos o subprocesso ou atividades de gerenciamento de configuração

Objetivo: controle da evolução de sistemas de software complexos

Configuração: conjunto de versões específicas de cada artefato ou item de configuração

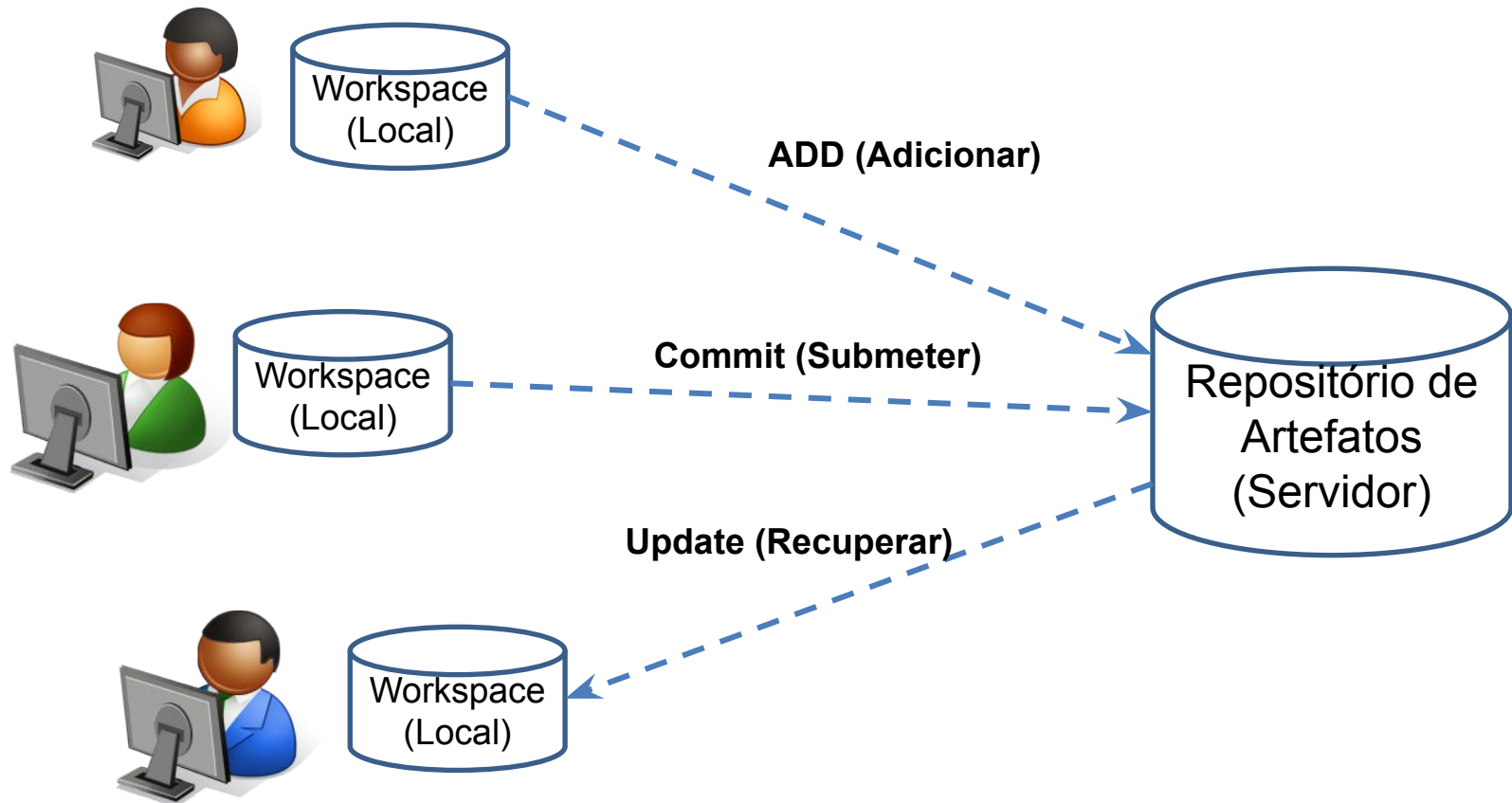


Gerência de Configuração

- Solicitação de Mudança
 - Dada uma **demanda** (feature nova, correção de bugs, melhorias, ...), é preciso que uma solicitação seja feita aos devidos responsáveis
 - Interface Biz-Dev
- Análise de Impacto
 - *Quanto tempo demora e custa para uma demanda ser entregue ao usuário final?*
 - Rastreabilidade
 - Continuidade de entrega
- Controle de Versão
 - Próximos slides...
- Construção do Produto (*Build*)
 - Gerenciamento de **dependências**
 - Automação
- Processo de Liberação (*Release*)
 - Pipeline de Implantação (*Deploy*)

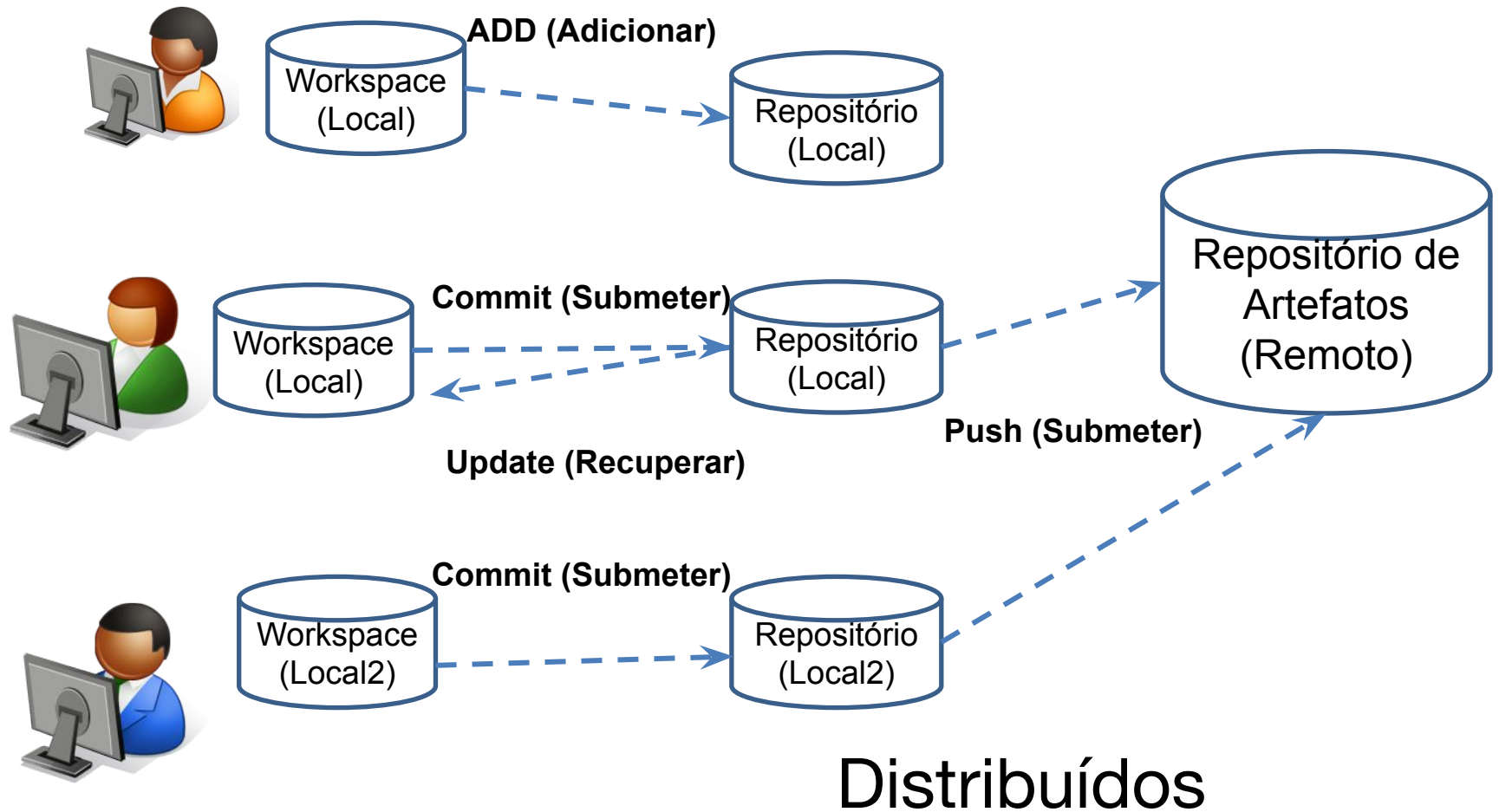


Controle de Versão de Artefatos



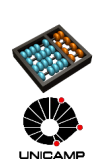
Cliente-Servidor

Controle de Versão de Artefatos



Conceitos (Locais)

- **Versão** (Revisão): Atributo que define o estado de um artefato após a criação ou após um conjunto de alterações
 - Em teoria “revisão” e “versão” são dois conceitos distintos, mas na prática podem ser utilizados da mesma forma.
- **Repositório**: Local (host/diretório) onde os artefatos são armazenados e mantidos sob controle de versão.
- **Workspace**: Cópia local (perspectiva do usuário) do repositório de artefatos, cujas versões podem ou não estar sincronizadas ao repositório.



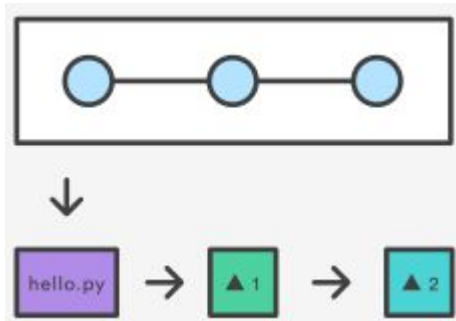
Conceitos (Operações)

- **Add** (Compartilhar): Estabelecer que um determinado artefato está sob controle de versão.
- **Commit** (Check-in): Submeter uma nova versão de um artefato ao repositório.
- **Update** (Check-out): Recuperar uma versão de um artefato do repositório.
- **Push**: Enviar alterações de um repositório para outro (remoto)
- **Pull**: Trazer alterações para um repositório a partir de outro remoto
- **Histórico** (*Log*): Conjunto de revisões de um artefato ordenado pelas datas de cada alteração.

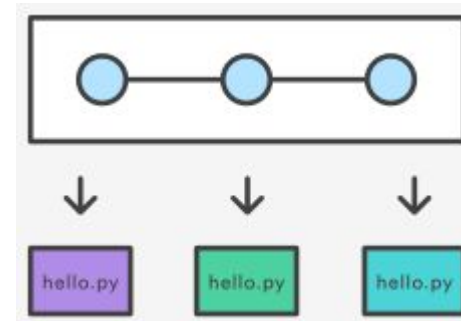
Ferramental de Apoio



Modelo de Versionamento



Deltas



C  pias

Git

Iniciando um Repositório

`git init <directory>`

- Cria um novo repositório (estrutura) git no diretório local ou especificado (*workspace*)
 - Subdiretório `.git`

```
breno@FRANCA MINGW32 ~/Workspaces
$ mkdir dfpage

breno@FRANCA MINGW32 ~/Workspaces
$ cd dfpage/

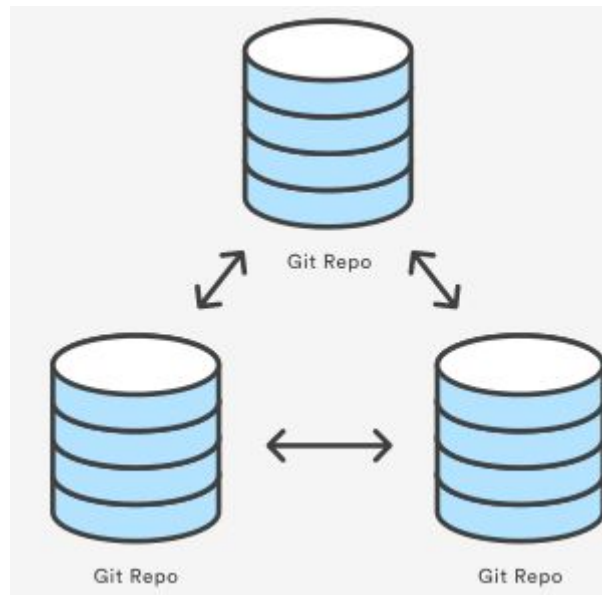
breno@FRANCA MINGW32 ~/Workspaces/dfpage
$ git init
Initialized empty Git repository in C:/Users/breno/Workspaces/dfpage/.git/

breno@FRANCA MINGW32 ~/Workspaces/dfpage (master)
$ ls -al
total 4
drwxr-xr-x 1 breno 197609 0 Mar 10 09:32 ./
drwxr-xr-x 1 breno 197609 0 Mar 10 09:31 ../
drwxr-xr-x 1 breno 197609 0 Mar 10 09:32 .git/
```



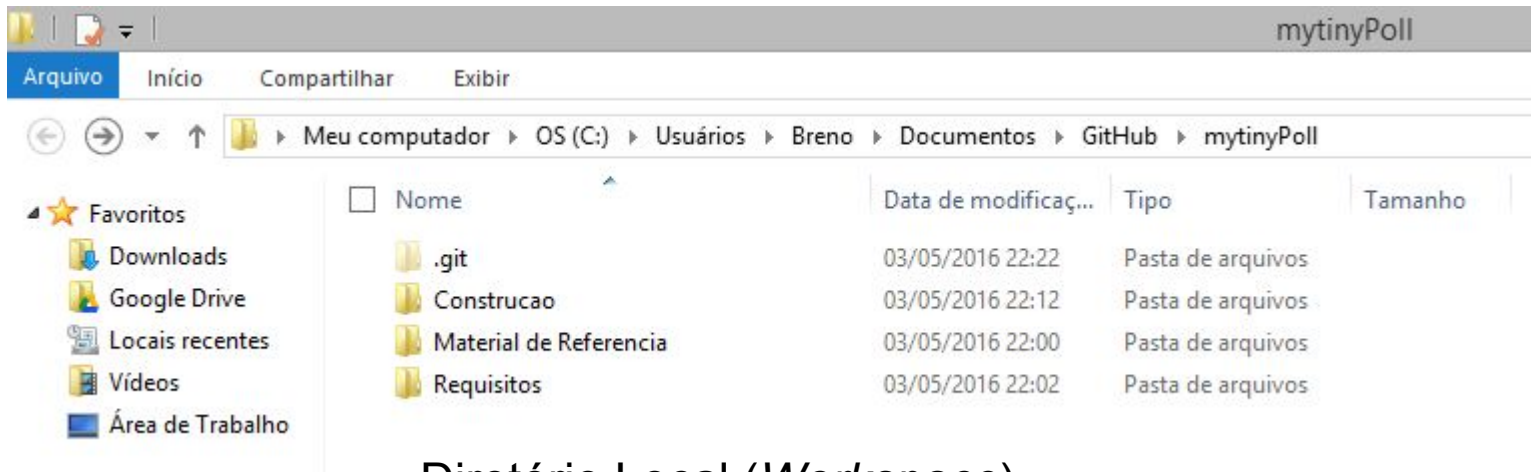
Iniciando um Repositório

- `git clone <repo url>`
 - Copia um repositório existente para o workspace



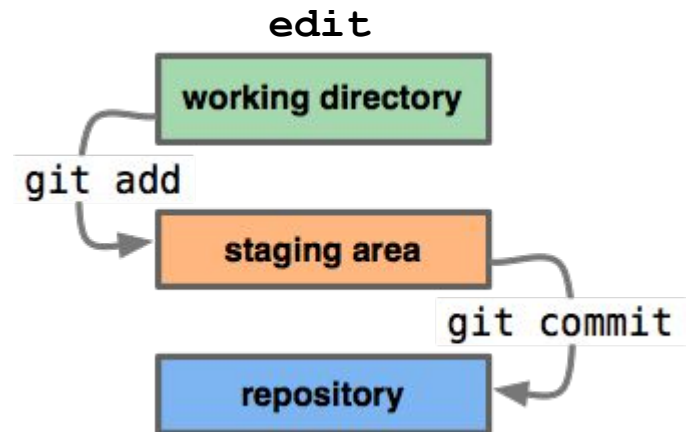
- `git config`
 - Configurações de usuários e repositório

Adicionando Artefatos



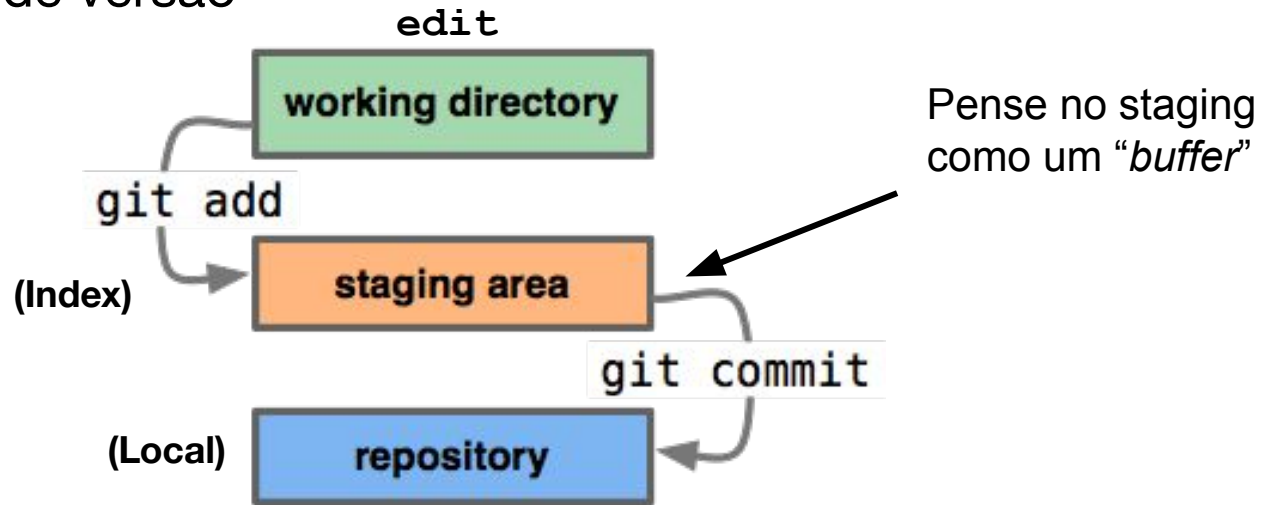
Diretório Local (*Workspace*)

Sempre que há uma edição no diretório, deve haver um `git add` e, possivelmente, um `git commit`



Adicionando Artefatos

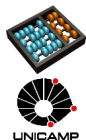
- `git add <file> or <directory>`
 - Adiciona arquivo ou diretório localizado no *workspace* ao controle de versão



- A staging area permite agrupar logicamente os commits
- `git commit`
 - Submete o conteúdo na staging area para o repositório de controle de versão (local)

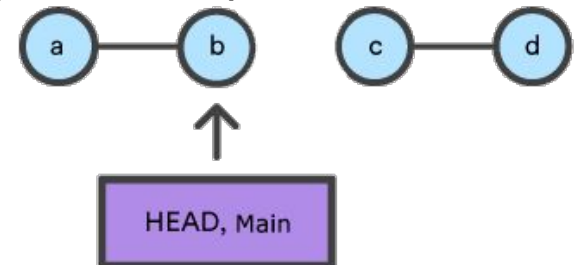
Adicionando Artefatos

- `git commit`
 - Submete o conteúdo na staging area para o repositório de controle de versão (local)
 - Parâmetros úteis:
 - m: inserir mensagem (entre aspas) de comentário
 - Ex.: `git commit -m "Minha mensagem"`
 - a: considerar todo conteúdo modificado do repositório



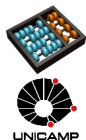
Recuperando Versões

- `git checkout <commit>`
 - Volta a versão para o commit especificado, sem alterar a árvore de versões
- `git revert`
 - Desfaz o efeito de um commit, gerando um novo commit
- `git reset`
 - Volta o estado do repositório local para uma posição especificada da árvore, mas pode deixar commits órfãos.

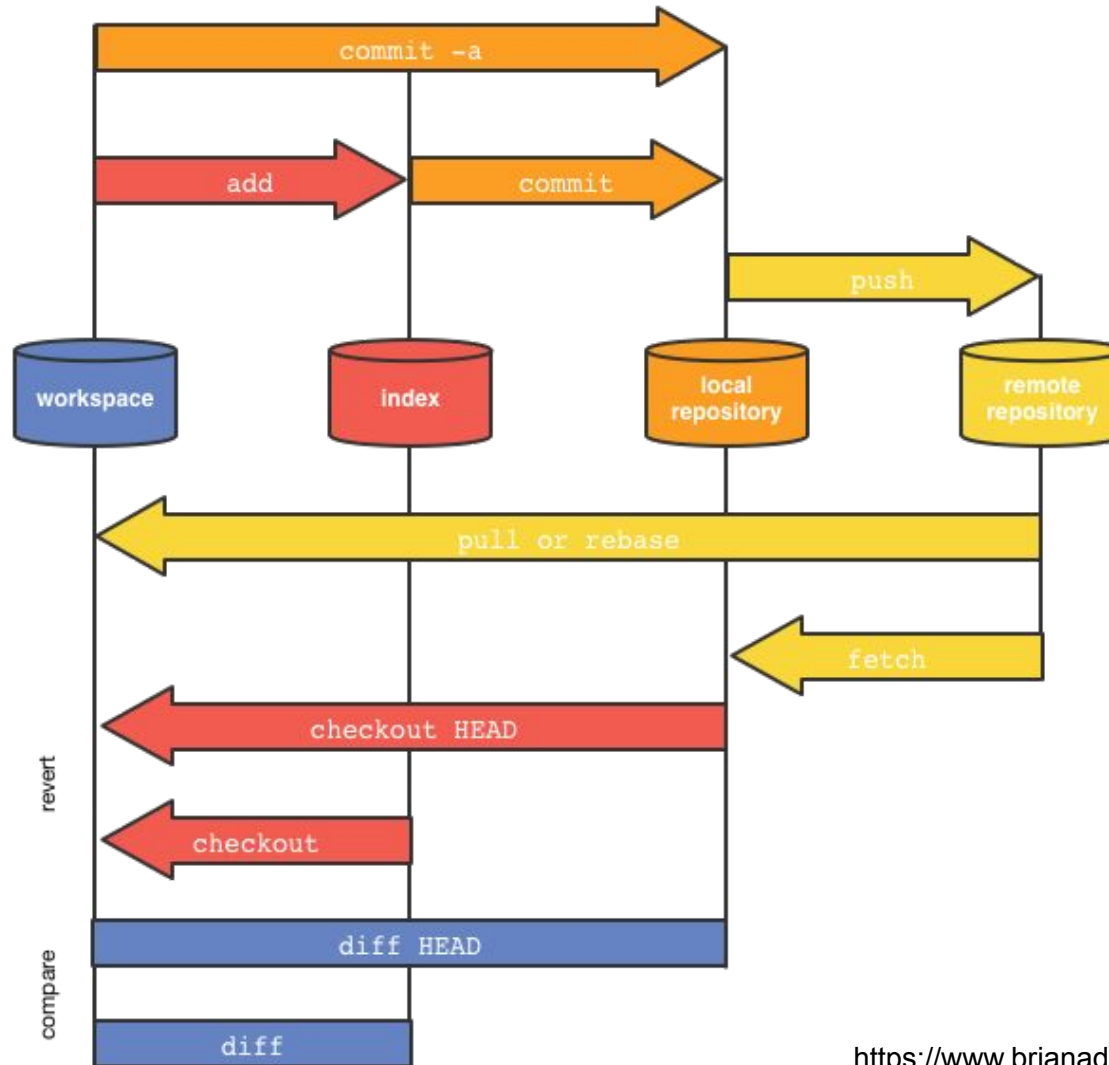


Repositórios Remotos

- `git fetch`
 - Importa *commits* de um repositório remoto para o local
 - `git fetch <remote repository name>`
- `git pull`
 - Sincroniza com repositório remoto recuperando mudanças
- `git push`
 - Sincroniza com repositório remoto enviando mudanças



Comandos Git de Transporte de Dados



Visualizando Histórico

- `git status`
 - Mostra versão atual da árvore de versões
- `git log`
 - Mostra uma lista em ordem decrescente de data dos commits

```
breno@unicamp:~/Workspace/tma-framework/tma-framework-a$ git log
commit ff5dc8f19f5ee8fff52fe40699a50f9ed5d48b99
Author: JorgeLuiz <jre-jl@hotmail.com>
Date:   Mon Apr 15 11:34:11 2019 -0300

    Updating Analyzer versions [analyzer integration ic cloud] (#82)

    * Updating Analyzer versions, including Spring Boot (now it is 2.1.3).

    * Updating readme file

    * Updating readme file

    * Updating readme file

    * Updating readme file

    * Updating readme file

commit 4bd828b78b04651f5a78c3fad221e0e816eb4167
Author: JorgeLuiz <jre-jl@hotmail.com>
Date:   Mon Feb 18 16:13:14 2019 -0300
:...skipping...
commit ff5dc8f19f5ee8fff52fe40699a50f9ed5d48b99
Author: JorgeLuiz <jre-jl@hotmail.com>
Date:   Mon Apr 15 11:34:11 2019 -0300

    Updating Analyzer versions [analyzer integration ic cloud] (#82)

    * Updating Analyzer versions, including Spring Boot (now it is 2.1.3).

    * Updating readme file

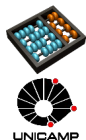
    * Updating readme file

    * Updating readme file

    * Updating readme file

    * Updating readme file

commit 4bd828b78b04651f5a78c3fad221e0e816eb4167
Author: JorgeLuiz <jre-jl@hotmail.com>
Date:   Mon Feb 18 16:13:14 2019 -0300
```

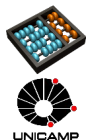


Removendo Artefatos

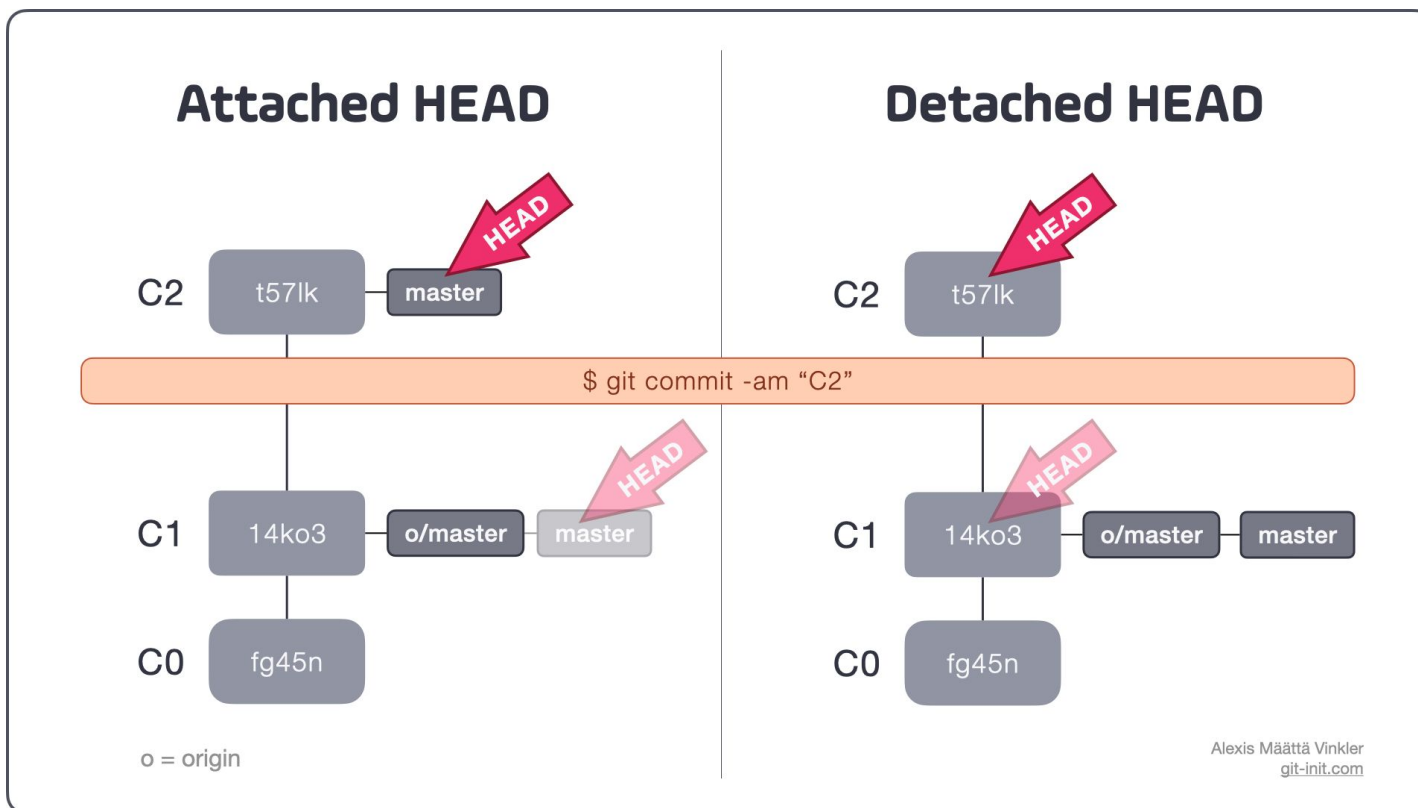
- `git clean`: remove arquivos não controlados da árvore
- **Retirar do controle de versão**
 - Arquivo `.gitignore`
 - Arquivo que descreve arquivos a serem ignorados pelo controle de versão:
 - Explicitamente
 - Padrão de formato

```
# Arquivos a serem ignorados  
Material de Referencia\aula.pdf  
Requisitos\Checklist\  
*.ini
```

Mais sintaxe em: git-scm.com/docs/gitignore



Precisamos falar sobre HEAD

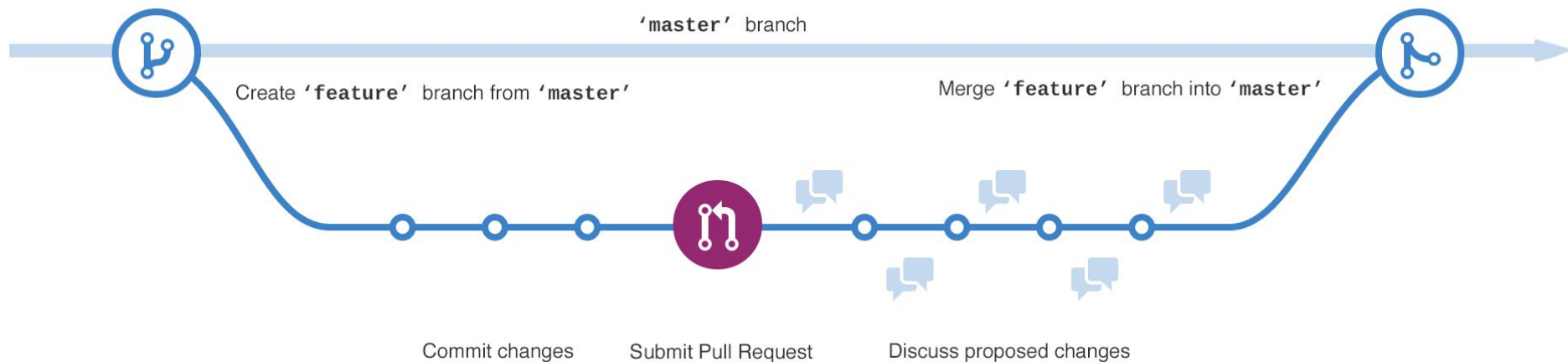


Fonte: Alexis Määttä Vinkler. What is HEAD in Git? <https://blog.git-init.com/what-is-head-in-git/>

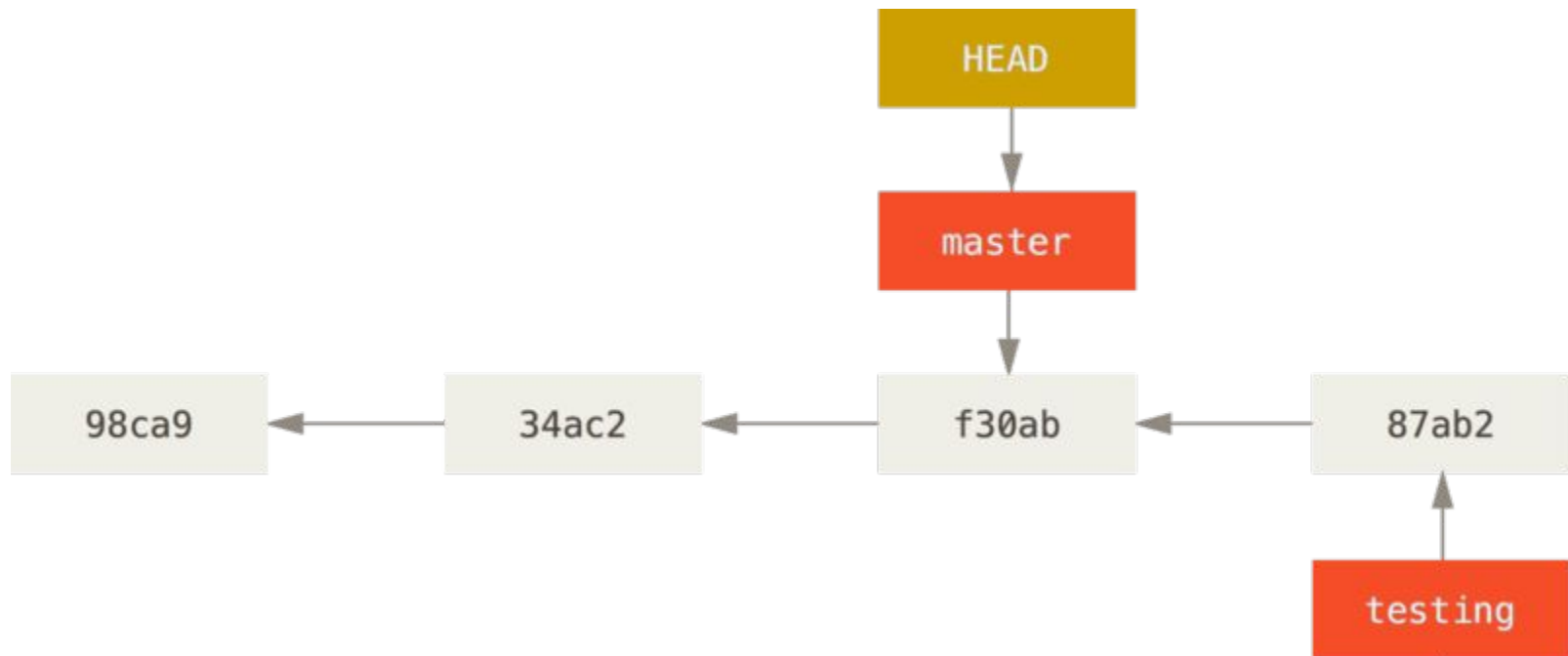
HEAD é uma referência para a versão atual com a qual você está trabalhando

Separando e Unificando

Branch e Merge de versões



HEAD



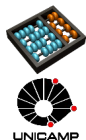
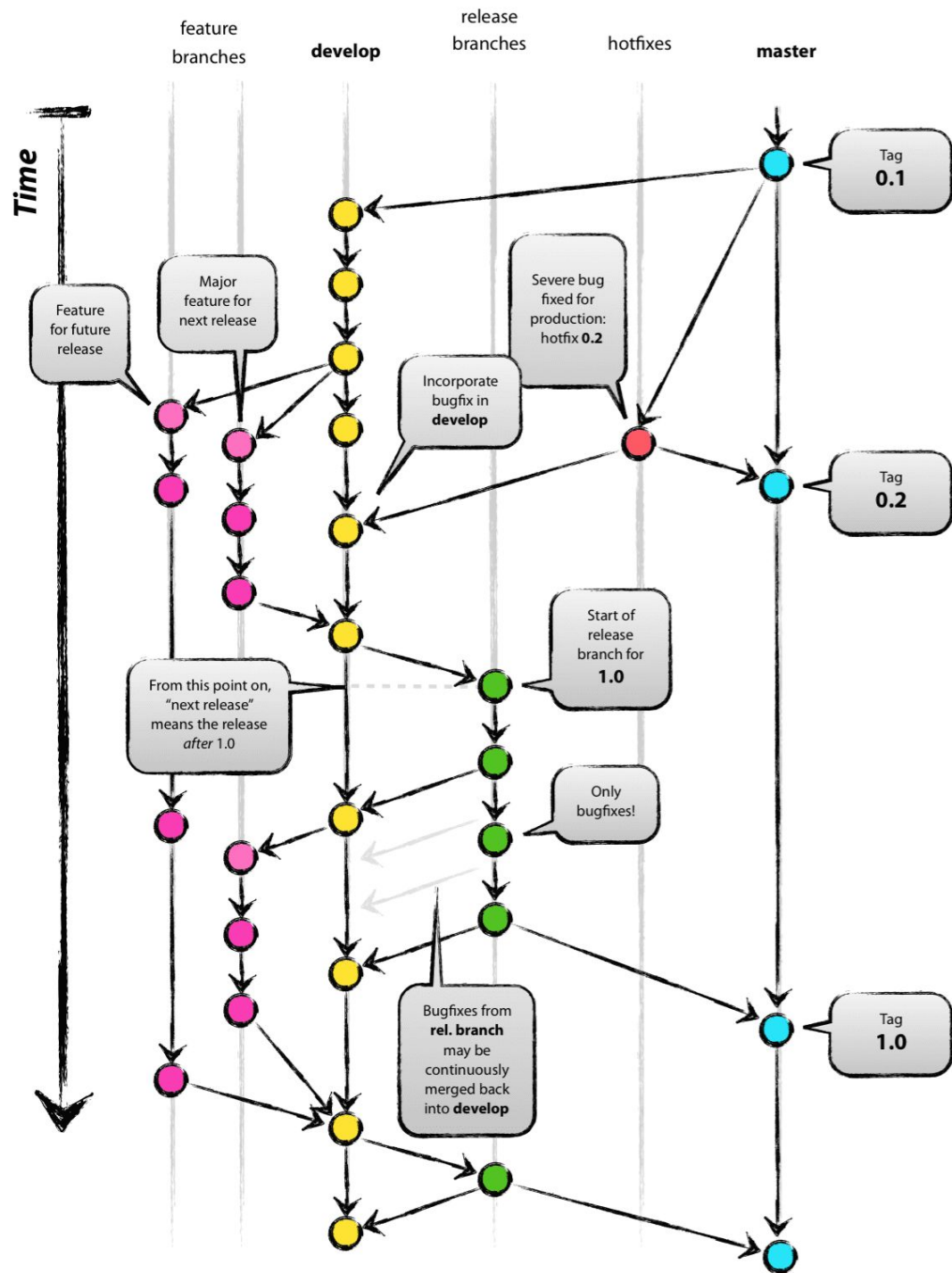
Branch per Feature

Raciocínio simples:

1. Para cada feature, cria-se uma branch nova.
2. Implementações dessa feature são feitas nessa branch.
3. Ao final, é integrada à branch principal

Gitflow

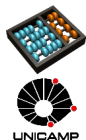
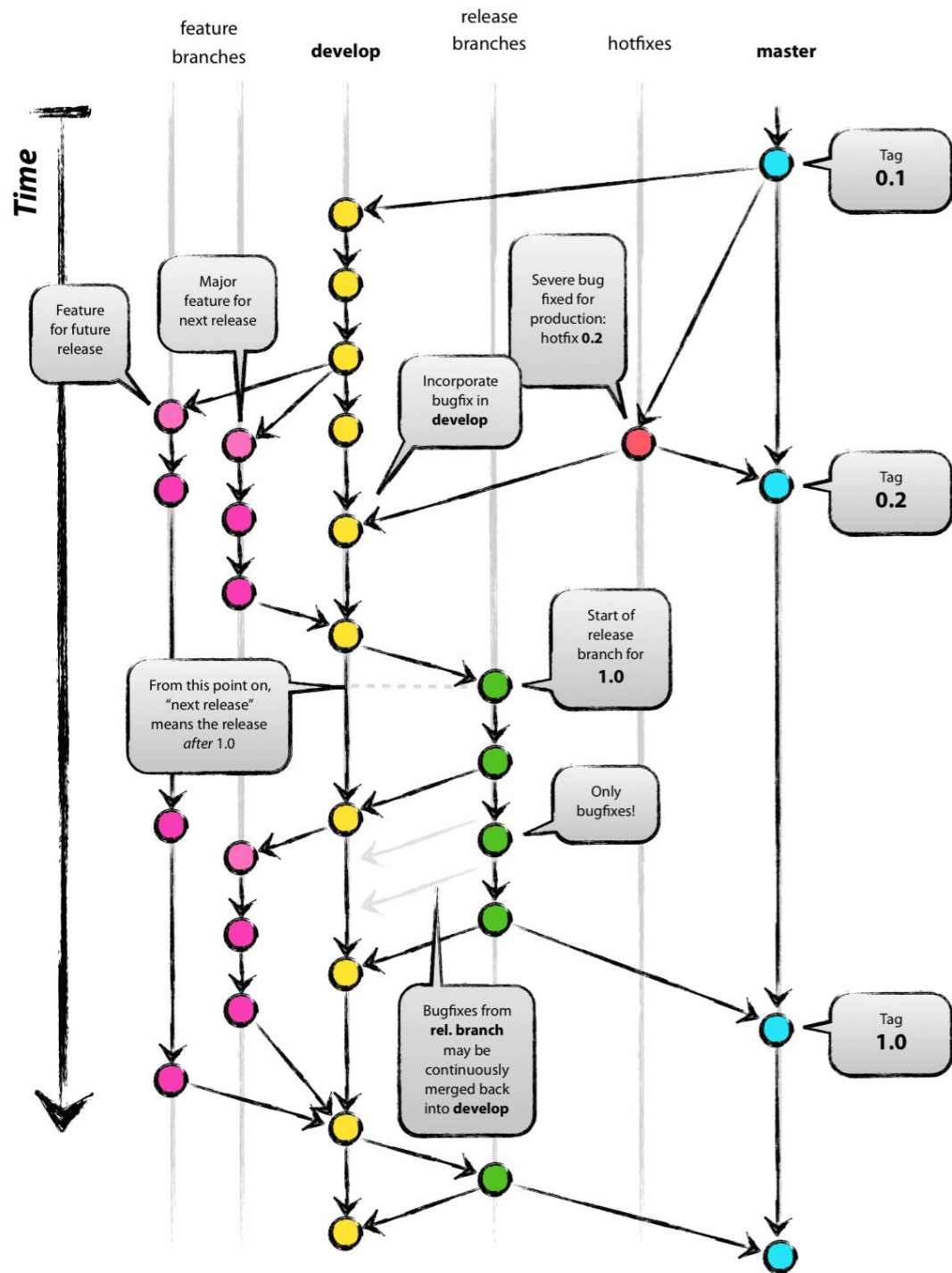
- master
- develop
- feature branch
- release branch



Branch per Feature

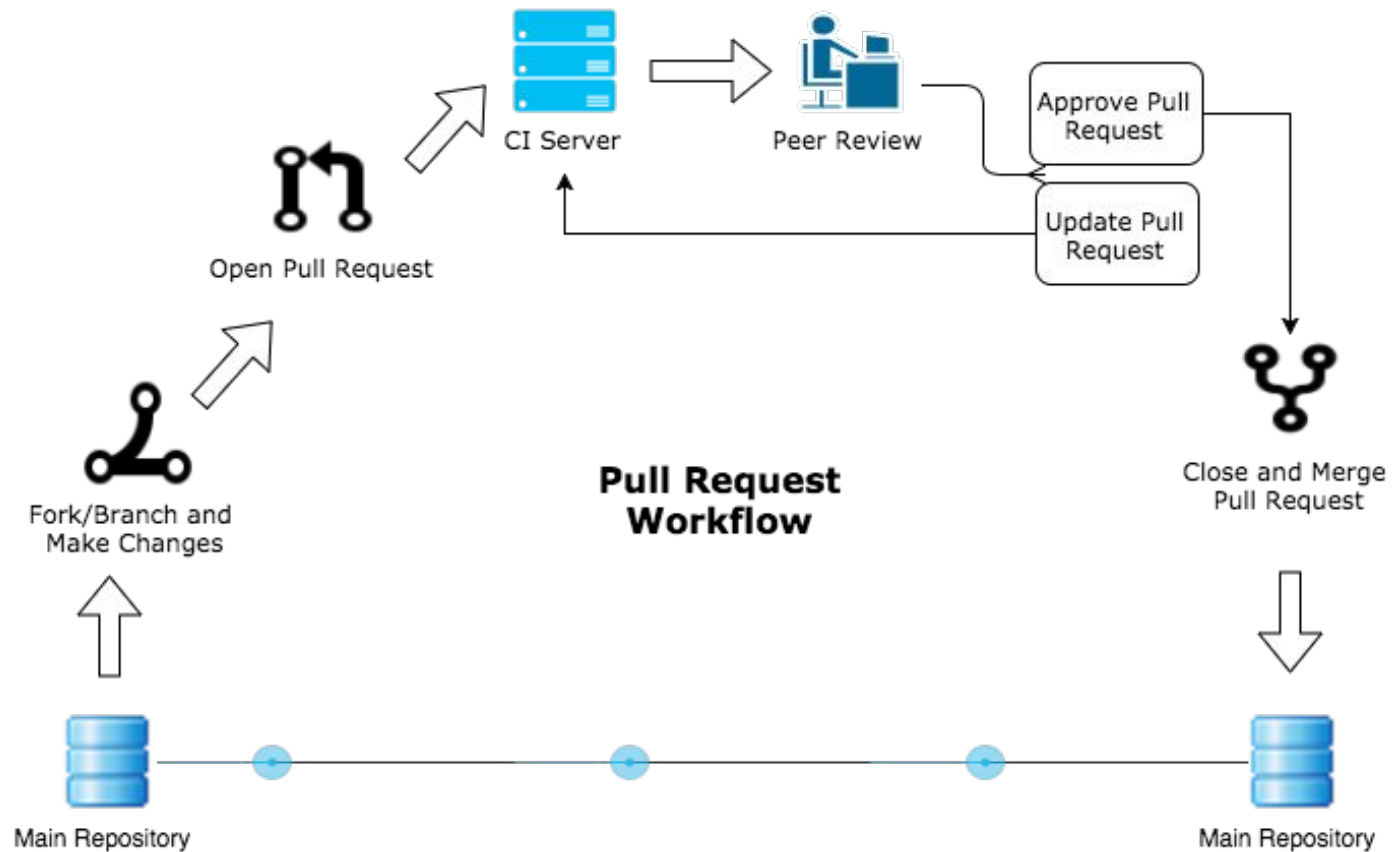
Processo fim-a-fim:

1. Abre uma issue com a nova feature
2. Cria uma branch nova a partir de **develop**
3. Desenvolve a feature na branch nova
4. Abre um pull request da nova branch para **develop**
5. Revisa as alterações realizadas
6. Integra (merge) a branch nova na **develop**
7. Fecha a issue referenciando o pull request



Pull/Merge Requests

Notificar à equipe que um trabalho está pronto e precisa ser integrado na branch principal (master)



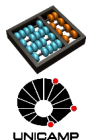
Tagging

- Referências que apontam para **pontos específicos** no histórico
 - **Como uma configuração!**
- Geralmente, utilizada como uma “foto” no histórico para marcar uma **release** (ex., v1.2.1)
- Metáfora: um branch que não muda

- `git tag`

```
$ git checkout master
Switched to branch 'master'
$ git merge --no-ff release-1.2
Merge made by recursive.
(Summary of changes)

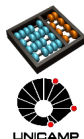
$ git tag -a 1.2
```



Controle de Versão

Boas práticas

- Manter sempre o repositório atualizado (sincronizado)
 - Atualizar (*sync*) os arquivos **antes** de utilizar e/ou modificar
- Inserir comentário listando as alterações realizadas
- Não adicionar itens que não pertençam ao projeto
- Não realizar “testes” no repositório do projeto
- Ao realizar *branches* e *merges*, ou ainda grandes modificações em arquivos compartilhados, comunicar (*Pull Request*) aos interessados **antes** de realizar a operação
- A cada novo release, criar um *branch* e realizar o *merge* com a main ao final do release
- A *branch* main deve SEMPRE estar com uma versão funcional!
- Cada macaco no seu galho! Ops... Cada sub-equipe (ou membro) com seu *branch*!



Gerenciamento de Configuração

Breno de França